DBMS -- ER Model (Entity-Relationship Model)

Target: Explain ER Model components verbally in 40s. Know all 4 attribute types + 4 cardinalities cold.

What Is the ER Model?

ER = Entity-Relationship
It is a high-level conceptual data model -- used to design a database before building it
Invented by Peter Chen (1976)
Think of it as the blueprint of a database -- drawn as an ER Diagram

Example: Designing a school database   you draw Students, Courses, Teachers as entities, then connect them with relationships before writing a single line of SQL.

ER Model -- 3 Main Components
ER Model
          / | \
        Entity Attribute Relation(ship)
          | |
      Weak   Key / Composite /
      Entity   Multivalued / Derived

1. ENTITY
An entity is any real-world object, class, person, or place that has data stored about it.
Examples: Manager, Product, Employee, Student, Department
Notation: Rectangle in ER diagram
```
Employee          Department
```
Weak Entity
A weak entity is one that cannot exist independently -- it depends on a parent (strong) entity.

It has no primary key of its own
It uses a partial key (discriminator) combined with the parent's PK
Notation: Double rectangle

Example: Installment depends on Loan -- an installment can't exist without a loan.
```
Loan     (relationship)   Installment     weak entity (double border)
```
Another example: OrderItem depends on Order. {OrderID + ItemNo} = composite key.

2. ATTRIBUTE
An attribute describes a property of an entity.
Examples: id, age, name, contact_number, phone_no
Notation: Oval in ER diagram
There are 4 types of attributes:

(a) Key Attribute

Used to uniquely identify each entity instance
Represents the Primary Key
Notation: Underlined text inside oval

(__id__)   underlined = key attribute
  /
Student

(b) Composite Attribute

An attribute made up of multiple sub-attributes
Notation: Oval with child ovals branching from it

Example: name is composed of first_name, middle_name, last_name
name
        / | \
  first mid last
  _name _name _name

Interview one-liner: "Composite attribute = one attribute split into smaller meaningful parts."

(c) Multivalued Attribute

An attribute that can hold more than one value for a single entity
Notation: Double oval

Example: A student can have multiple phone numbers   phone_no is multivalued
((phone_no))   double oval = multivalued

In relational implementation: multivalued attributes are stored in a separate table.

(d) Derived Attribute

An attribute whose value can be calculated from another attribute
Notation: Dashed oval

Example: age can be derived from Birth_Date (calculated at runtime)
(- - age - -)   dashed oval = derived

In relational implementation: derived attributes are often not stored -- computed by query.

Attribute Types -- Summary Table

Type	Meaning	ER Symbol	Example
Simple	Single, indivisible value	Plain oval	age, salary
Key	Uniquely identifies entity	Underlined oval	student_id
Composite	Made of sub-attributes	Oval + sub-ovals	name = first + last
Multivalued	Holds multiple values	Double oval	phone_no
Derived	Calculated from another attr	Dashed oval	age from DOB

3. RELATIONSHIP
A relationship describes the association between two or more entities.

Notation: Diamond (rhombus) in ER diagram
Named with a verb

```
Teacher      Teaches          Department
```

Cardinality -- 4 Types of Relationships

Cardinality = how many instances of one entity relate to how many of another.

(a) One-to-One (1:1)
One instance of Entity A relates to exactly one instance of Entity B.
Example: A female can marry one male; a male can marry one female.
Female 1 married to 1 Male
Other examples: Person Passport, Employee Company Car

(b) One-to-Many (1:M)
One instance of Entity A relates to many instances of Entity B.
Example: A scientist can invent many inventions, but each invention is by only one specific scientist.
Scientist 1 invents M Invention
Other examples: Department Employees, Author Books

(c) Many-to-One (M:1)
Many instances of Entity A relate to one instance of Entity B.
(This is the reverse of 1:M -- depends on which direction you read it.)
Example: Many students enroll in one course, but a student enrolls in only one course.
Student M enroll 1 Course

(d) Many-to-Many (M:M)
Many instances of Entity A relate to many instances of Entity B.
Example: An employee can work on many projects; a project can have many employees.
Employee M is assigned M Project
In relational DB: M:M relationships are implemented using a junction/bridge table.
sql
```
-- Junction table for Employee-Project
CREATE TABLE emp_project (
  emp_id     INT REFERENCES employees(emp_id),
  project_id INT REFERENCES projects(project_id),
  PRIMARY KEY (emp_id, project_id)
);
```

ER Diagram Notation -- Line Symbols

Symbol	Meaning
1	Exactly one
M / <	Many
1+	One or more
0/1	Zero or one
0+	Zero or many

Example from ebook -- Company ER:
Company Employee Projects
(one company (one or more) (zero or many
has employees) per employee)

Integrity Constraints
Integrity constraints = rules that ensure data in the DB is valid, accurate, and consistent.
```
Integrity Constraint
           /                  \
   Domain Constraint        Key Constraint
                          /           \
```

Entity Integrity   Referential Integrity

1. Domain Constraint
Rule: Every attribute value must belong to its defined domain (valid set of values).
Example: AGE is defined as an integer. Inserting 'A' violates the domain constraint.

ID
NAME
SEMESTER
AGE

1000
Tom
1st
17

1001
Johnson
2nd
24

1002
Morgan
8th
A   domain violation

2. Entity Integrity Constraint
Rule: The Primary Key of any table cannot be NULL.
Every row must be uniquely identifiable -- a NULL PK makes identification impossible.
Example:

Emp_ID
Emp_NAME
SALARY

123
Jak
30000

164
John
20000

NULL
Jackson
27000   PK cannot be NULL

3. Referential Integrity Constraint
Rule: A Foreign Key value must always match an existing Primary Key in the referenced table -- or be NULL.
You cannot reference a row that doesn't exist.
Example:

EMP_ID

EMP_NAME
DEPT_ID

01
Rahul
D01

02
Kumar
D02

03
Raj
D99 D99 doesn't exist in Department table

Rule also states: FK and the PK it references must have the same data type.

4. Key Constraint
Rule: All values in the Primary Key column(s) must be unique -- no duplicate rows allowed.
Example:

ID
NAME
SEMESTER
AGE

1000
Tom
1st
17

1001
Johnson
2nd
24

1002
Kate
3rd
21

1001
Morgan
8th
22 duplicate ID

Integrity Constraints -- Quick Reference

Constraint
Rule
What it prevents

Domain
Values must be from valid domain
Wrong data type (e.g., text in age)

Entity Integrity
PK cannot be NULL
Unidentifiable rows

Referential Integrity
FK must match existing PK
Orphan records, broken links

Key Constraint
PK values must be unique
Duplicate rows

Your 40-Second Script -- ER Model

"The ER Model is a high-level conceptual design tool used to plan a database before building it. It has three components -- entities, attributes, and relationships. An entity is a real-world object like Student or Employee, shown as a rectangle. Attributes describe properties -- they can be simple, composite, multivalued, or derived, each with a different oval notation. Relationships, shown as diamonds, describe associations between entities with cardinality: one-to-one, one-to-many, many-to-one, or many-to-many."

Follow-Up Questions (Expect These)
Q: What is the difference between an entity and an attribute?

An entity is a real-world object that has its own existence (Student, Department). An attribute is a property of that entity (name, age). If something has its own sub-properties and existence, it's likely an entity, not an attribute.

Q: How do you implement a Many-to-Many relationship in a relational DB?

Using a junction/bridge table that holds the FKs of both entities as a composite PK. Example: enrollment(student_id, course_id).

Q: What is a partial key / discriminator?

The attribute of a weak entity that, combined with the parent entity's PK, uniquely identifies it. Example: ItemNumber in OrderItem -- meaningful only alongside OrderID.

Q: Difference between Entity Integrity and Referential Integrity?

Entity Integrity: PK of a table cannot be NULL. Referential Integrity: FK must reference a valid existing PK (no dangling references).